

Flutter API Integration Handbook

71 endpoints · 16 modules · API v1

BASE URL

`https://YOUR_API_HOST/api/v1`

Default JSON header: Content-Type: application/json

Protected header: Authorization: Bearer <accessToken>

Generated from current Express routes, validation, and controllers

How to use this handbook

This document covers every currently mounted HTTP endpoint. Replace `:id`, `:chatRoomId`, and `:notificationId` with actual identifiers. Examples shorten MongoDB documents to their mobile-relevant fields.

Standard success envelope

```
JSON
{
  "success": true,
  "message": "Operation completed",
  "data": {
    "id": "...",
  }
}
```

Standard error envelope

```
JSON
{
  "success": false,
  "message": "Human-readable error",
  "errorMessages": [
    {
      "path": "fieldName",
      "message": "Detailed error"
    }
  ]
}
```

Pagination

List endpoints commonly accept `page`, `limit`, `sortBy`, `sortOrder`, `searchTerm`, and model-specific filters. The response includes pagination when shown in the endpoint example.

Files

For multipart requests, do not manually force the multipart boundary in Flutter. Let Dio/http MultipartRequest build Content-Type. Fields named `data` contain a JSON-encoded string where documented.

Role labels

USER is the regular mobile user. ADMIN and SUPER_ADMIN routes must only appear in authorized admin builds. Apple server-only routes must never be invoked by Flutter.

Module index

System: 1 endpoint

Authentication: 7 endpoints

Users: 6 endpoints

Google OAuth: 5 endpoints

Income: 5 endpoints

Expense: 5 endpoints

Receipt OCR and Scan: 3 endpoints

Audit Risk: 1 endpoint

Bank Transactions: 4 endpoints

Chat: 5 endpoints

Reports: 3 endpoints

Notices: 3 endpoints

Terms and Conditions: 5 endpoints

Privacy Policy: 5 endpoints

Apple Subscription: 8 endpoints

Administration: 5 endpoints

System

1 endpoint in this module.

1. GET / (outside /api/v1) — Basic server health page.

1. GET

/ (outside /api/v1)

Basic server health page.

Access: Public

Request

No request body.

Success response

HTTP

200 text/html server-alive page

Integration note

Use the API hostname root, for example `https://YOUR_API_HOST/`. This endpoint returns HTML, not JSON.

Authentication

7 endpoints in this module.

2. POST /auth/login — Authenticate with email/password.
3. POST /auth/forget-password — Send a password-reset OTP.
4. POST /auth/verify-email — Verify registration OTP or password-reset OTP.
5. POST /auth/reset-password — Set a new password after reset OTP verification.
6. POST /auth/change-password — Change the authenticated user password.
7. POST /auth/resend-otp — Resend an email OTP.
8. POST /auth/refresh-token — Rotate access and refresh tokens.

2. POST

/auth/login

Authenticate with email/password.

Access: Public

Request

JSON / QUERY

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "User logged in successfully.",
  "data": {
    "accessToken": "<jwt>",
    "refreshToken": "<jwt>"
  }
}
```

3. POST

/auth/forget-password

Send a password-reset OTP.

Access: Public

Request

JSON / QUERY

```
{
  "email": "user@example.com"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Please check your email. We have sent you a one-time passcode (OTP).",
  "data": {
    "id": "..."
  }
}
```


4. POST

/auth/verify-email

Verify registration OTP or password-reset OTP.

Access: Public

Request

JSON / QUERY

```
{
  "email": "user@example.com",
  "oneTimeCode": 123456
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Email verify successfully",
  "data": null
}
```

5. POST

/auth/reset-password

Set a new password after reset OTP verification.

Access: Reset token in Authorization header

Request

JSON / QUERY

```
{
  "newPassword": "newPassword123",
  "confirmPassword": "newPassword123"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Your password has been successfully reset.",
  "data": {
    "id": "..."
  }
}
```

Integration note

Authorization header contains the reset token returned by verify-email; this is not an access JWT.

6. POST

/auth/change-password

Change the authenticated user password.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "currentPassword": "oldPassword123",
  "newPassword": "newPassword123",
  "confirmPassword": "newPassword123"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Your password has been successfully changed",
  "data": {
    "id": "..."
  }
}
```

7. POST

/auth/resend-otp

Resend an email OTP.

Access: Public

Request

JSON / QUERY

```
{
  "email": "user@example.com"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "OTP resent successfully, please check your email"
}
```

8. POST

/auth/refresh-token

Rotate access and refresh tokens.

Access: Public

Request

JSON / QUERY

```
{
  "refreshToken": "<refresh-jwt>"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Token refreshed successfully.",
  "data": {
    "createToken": "<new-access-jwt>",
    "refreshToken": "<new-refresh-jwt>"
  }
}
```

Integration note

The access token property is currently named createToken.

Users

6 endpoints in this module.

- 9. POST /user — Register a local user.
- 10. GET /user/profile — Get authenticated profile.
- 11. PATCH /user/profile — Update profile and optional image.
- 12. DELETE /user/profile — Delete the authenticated account.
- 13. GET /user — List users.
- 14. PATCH /user/:id/status — Activate or block a user.

9. POST

/user

Register a local user.

Access: Public

Request

JSON / QUERY

```
{
  "name": "Jane Doe",
  "contact": "+15555550100",
  "email": "jane@example.com",
  "password": "password123"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "id": "...",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "verified": false
  }
}
```

10. GET

/user/profile

Get authenticated profile.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Profile data retrieved successfully",
  "data": {
    "id": "...",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "plan": "Free",
    "isPremium": false
  }
}
```


11. PATCH

/user/profile

Update profile and optional image.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\"name\":\"Jane Doe\"}",
    "image": "<image file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": "...",
    "name": "Jane Doe",
    "image": "https://..."
  }
}
```

Integration note

Use multipart/form-data. data is a JSON string; image is optional.

12. DELETE

/user/profile

Delete the authenticated account.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Account deleted successfully",
  "data": {
    "id": "...
  }
}
```

13. GET

/user

List users.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, searchTerm, sortBy, sortOrder, and model fields"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Users retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

14. PATCH

/user/:id/status

Activate or block a user.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "status": "block"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "User status updated successfully",
  "data": {
    "id": "...",
    "status": "block"
  }
}
```

Google OAuth

5 endpoints in this module.

- 15. GET /oauth/google — Start Google OAuth in a browser/webview.
- 16. GET /oauth/google/callback — Google returns here; backend redirects to Flutter/web callback.
- 17. GET /oauth/profile — Return JWT profile claims.
- 18. GET /oauth/status — Check configured OAuth providers.
- 19. GET /oauth/login-failed — OAuth failure landing endpoint.

15. GET

/oauth/google

Start Google OAuth in a browser/webview.

Access: Public

Request

No request body.

Success response

HTTP

302 Redirect to Google

/oauth/google/callback

Google returns here; backend redirects to Flutter/web callback.

[Access: Google OAuth callback](#)

Request

No request body.

Success response

HTTP

```
302 Redirect: FRONTEND_OAUTH_CALLBACK_URL?accessToken=...&refreshToken=...&userId=...
```

Integration note

Do not call manually. Register and handle the configured callback/deep link.

/oauth/profile

Return JWT profile claims.

Access: Bearer JWT

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "User profile retrieved successfully",
  "data": {
    "id": "...",
    "email": "user@example.com",
    "role": "USER"
  }
}
```


/oauth/status

Check configured OAuth providers.

Access: Public

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "OAuth provider status retrieved",
  "data": {
    "google": {
      "configured": true,
      "name": "Google"
    }
  }
}
```

/oauth/login-failed

OAuth failure landing endpoint.

Access: Public

Request

No request body.

Success response

JSON

```
{
  "success": false,
  "message": "OAuth login failed",
  "error": "Authentication was not successful"
}
```

Income

5 endpoints in this module.

- 20. GET /income — Monthly summary or detailed month records.
- 21. POST /income — Create income with optional document/image.
- 22. PATCH /income/:id — Update owned income.
- 23. DELETE /income/:id — Delete owned income.
- 24. GET /income/history — Paginated complete income history.

/income

Monthly summary or detailed month records.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "month=2026-08 OR month=8&year=2026; page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Income list retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "amount": 125.5,
      "category": "Salary",
      "date": "2026-08-03T00:00:00.000Z",
      "description": "Example",
      "fileUrl": null
    }
  ]
}
```

Integration note

Without month/year, data is [[year, month, total]] and pagination is omitted.

/income

Create income with optional document/image.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\\\"amount\\\":125.5,\\\"category\\\":\\\"Salary\\\",\\\"date\\\":\\\"2026-08-03\\\",\\\"description\\\":\\\"Example\\\"}",
    "doc": "<file>",
    "image": "<file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Income recorded successfully",
  "data": {
    "id": "...",
    "amount": 125.5,
    "category": "Salary",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": null
  }
}
```

Integration note

Use multipart/form-data; choose doc or image. JSON-only body is also accepted when no file is needed.

/income/:id

Update owned income.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\"amount\":150}",
    "doc": "<optional file>",
    "image": "<optional file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Income updated successfully",
  "data": {
    "id": "...",
    "amount": 150,
    "category": "Salary",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": null
  }
}
```

23. DELETE

/income/:id

Delete owned income.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Income deleted successfully",
  "data": {
    "id": "...
  }
}
```

/income/history

Paginated complete income history.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder, category, date"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Income history retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "amount": 125.5,
      "category": "Salary",
      "date": "2026-08-03T00:00:00.000Z",
      "description": "Example",
      "fileUrl": null
    }
  ]
}
```


Expense

5 endpoints in this module.

- 25. GET /expense — Monthly summary or detailed month records.
- 26. POST /expense — Create expense with optional document/image.
- 27. PATCH /expense/:id — Update owned expense.
- 28. DELETE /expense/:id — Delete owned expense.
- 29. GET /expense/history — Paginated complete expense history.

/expense

Monthly summary or detailed month records.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "month=2026-08 OR month=8&year=2026; page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense list retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "amount": 125.5,
      "category": "Food",
      "date": "2026-08-03T00:00:00.000Z",
      "description": "Example",
      "fileUrl": null
    }
  ]
}
```

Integration note

Without month/year, data is [{year, month, total}].

/expense

Create expense with optional document/image.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\"amount\":25.5,\"category\":\"Food\",\"date\":\"2026-08-03\"}",
    "doc": "<file>",
    "image": "<file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense recorded successfully",
  "data": {
    "id": "...",
    "amount": 25.5,
    "category": "Food",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": null
  }
}
```

/expense/:id

Update owned expense.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\"description\\\":\\\"Updated\\\"}\",
    "doc": "<optional file>",
    "image": "<optional file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense updated successfully",
  "data": {
    "id": "...",
    "amount": 125.5,
    "category": "Food",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": null
  }
}
```

/expense/:id

Delete owned expense.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Expense deleted successfully",
  "data": {
    "id": "...
  }
}
```

/expense/history

Paginated complete expense history.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder, category, date"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense history retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "amount": 125.5,
      "category": "Food",
      "date": "2026-08-03T00:00:00.000Z",
      "description": "Example",
      "fileUrl": null
    }
  ]
}
```

Receipt OCR and Scan

3 endpoints in this module.

30. POST /ocr/analyze — Analyze receipt image or supplied OCR text without saving an expense.

31. POST /scan/extract-review — OCR a receipt and create an expense draft/record.

32. PATCH /scan/:id — Correct OCR-created expense.

/ocr/analyze

Analyze receipt image or supplied OCR text without saving an expense.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "file": "<receipt image>",
    "text": "<alternative receipt text>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Receipt analyzed successfully",
  "data": {
    "amount": 42.75,
    "category": "Food"
  }
}
```

Integration note

Send either file or text.

/scan/extract-review

OCR a receipt and create an expense draft/record.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "file": "<receipt image>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense created successfully",
  "data": {
    "id": "...",
    "amount": 125.5,
    "category": "Food",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": "https://..."
  }
}
```

/scan/:id

Correct OCR-created expense.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "amount": 42.75,
  "category": "Food",
  "date": "2026-08-03",
  "description": "Lunch",
  "fileUrl": "https://...",
  "fileName": "receipt.jpg"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Expense updated successfully",
  "data": {
    "id": "...",
    "amount": 125.5,
    "category": "Food",
    "date": "2026-08-03T00:00:00.000Z",
    "description": "Example",
    "fileUrl": null
  }
}
```

Audit Risk

1 endpoint in this module.

33. GET /audit-risk — Count income/expense records without evidence files.

/audit-risk

Count income/expense records without evidence files.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Audit risk count retrieved successfully",
  "data": {
    "count": 3
  }
}
```

Bank Transactions

4 endpoints in this module.

- 34. POST /bank-transaction — Create an owned bank transaction.
- 35. GET /bank-transaction — List owned bank transactions.
- 36. PATCH /bank-transaction/:id — Update an owned bank transaction.
- 37. DELETE /bank-transaction/:id — Delete an owned bank transaction.

/bank-transaction

Create an owned bank transaction.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "amount": 125.5,
  "bankName": "Example Bank",
  "accountNumberLast4Digits": "1234",
  "refId": "REF-123",
  "date": "2026-08-03"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Bank transaction created successfully",
  "data": {
    "id": "...",
    "amount": 125.5,
    "bankName": "Example Bank",
    "accountNumberLast4Digits": "1234",
    "refId": "REF-123",
    "date": "2026-08-03T00:00:00.000Z"
  }
}
```

/bank-transaction

List owned bank transactions.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, searchTerm, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Bank transactions retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

/bank-transaction/:id

Update an owned bank transaction.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{  
  "amount": 150  
}
```

Success response

JSON

```
{  
  "success": true,  
  "message": "Bank transaction updated successfully",  
  "data": {  
    "id": "...",  
    "amount": 150  
  }  
}
```


/bank-transaction/:id

Delete an owned bank transaction.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Bank transaction deleted successfully",
  "data": {
    "id": "...
  }
}
```

Chat

5 endpoints in this module.

- 38. POST /chat/create-room — Create or return the user's support room.
- 39. GET /chat/my-rooms — List chat rooms visible to the user/admin.
- 40. POST /chat/send-message/:chatRoomId — Send text, image, or PDF.
- 41. GET /chat/:chatRoomId/messages — List room messages.
- 42. PATCH /chat/:chatRoomId/mark-read — Mark other participants' messages read.

/chat/create-room

Create or return the user's support room.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{}
```

Success response

JSON

```
{
  "success": true,
  "message": "Chat room created successfully",
  "data": {
    "id": "...",
    "participants": [
      "..."
    ],
    "admin": "...",
    "user": "..."
  }
}
```

/chat/my-rooms

List chat rooms visible to the user/admin.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Chat rooms retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

/chat/send-message/:chatRoomId

Send text, image, or PDF.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "data": "{\"messageType\":\"text\",\"content\":\"Hello\"}",
    "file": "<image/pdf when applicable>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Message sent successfully",
  "data": {
    "id": "...",
    "messageType": "text",
    "content": "Hello",
    "readBy": [
      "..."
    ]
  }
}
```

Integration note

Use JSON body without file, or multipart with data JSON string and file.

41. GET

/chat/:chatRoomId/messages

List room messages.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Chat messages retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

/chat/:chatRoomId/mark-read

Mark other participants' messages read.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Messages marked as read successfully",
  "data": {
    "id": "..."
  }
}
```

Reports

3 endpoints in this module.

43. GET /reports/pdf — Download financial report as PDF.

44. GET /reports/excel — Download report as XLSX.

45. GET /reports/csv — Download report as CSV.

43. GET

/reports/pdf

Download financial report as PDF.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{  
  "query": "startDate=2026-08-01&endDate=2026-08-31"  
}
```

Success response

HTTP

200 application/pdf binary

Integration note

Use Flutter byte download/share handling, not JSON decoding.

44. GET

/reports/excel

Download report as XLSX.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "startDate and endDate (optional ISO dates)"
}
```

Success response

HTTP

```
200 application/vnd.openxmlformats-officedocument.spreadsheetml.sheet binary
```

45. GET

/reports/csv

Download report as CSV.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "startDate and endDate (optional ISO dates)"
}
```

Success response

HTTP

200 text/csv binary

Notices

3 endpoints in this module.

46. POST /notices — Create notice with document.

47. GET /notices — List notices.

48. DELETE /notices/:id — Delete notice and document.

/notices

Create notice with document.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

MULTIPART / QUERY

```
{
  "multipart": {
    "type": "IRS Notice | Case Status",
    "doc": "<file> OR document=<file>"
  }
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Notice created successfully",
  "data": {
    "id": "...",
    "type": "IRS Notice",
    "document": "https://..."
  }
}
```

/notices

List notices.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder, type"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Notices retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

48. DELETE

/notices/:id

Delete notice and document.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Notice deleted successfully",
  "data": {
    "id": "...
  }
}
```

Terms and Conditions

5 endpoints in this module.

- 49. POST /terms-and-conditions — Create terms.
- 50. GET /terms-and-conditions — List terms.
- 51. GET /terms-and-conditions/:id — Get one terms document.
- 52. PATCH /terms-and-conditions/:id — Update terms.
- 53. DELETE /terms-and-conditions/:id — Delete terms.

/terms-and-conditions

Create terms.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "title": "Terms",
  "description": "...",
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Terms and Conditions created successfully",
  "data": {
    "id": "...",
    "title": "Terms",
    "description": "...",
  }
}
```

/terms-and-conditions

List terms.

Access: Public

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Terms and Conditions retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

51. GET

/terms-and-conditions/:id

Get one terms document.

Access: Public

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Terms and Conditions retrieved successfully",
  "data": {
    "id": "...",
    "title": "Terms",
    "description": "..."
  }
}
```

/terms-and-conditions/:id

Update terms.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "title": "Updated Terms"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Terms and Conditions updated successfully",
  "data": {
    "id": "...",
    "title": "Updated Terms"
  }
}
```

/terms-and-conditions/:id

Delete terms.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Terms and Conditions deleted successfully",
  "data": {
    "id": "...
  }
}
```

Privacy Policy

5 endpoints in this module.

- 54. POST /privacy-policy — Create privacy policy.
- 55. GET /privacy-policy — List privacy policies.
- 56. GET /privacy-policy/:id — Get one privacy policy.
- 57. PATCH /privacy-policy/:id — Update privacy policy.
- 58. DELETE /privacy-policy/:id — Delete privacy policy.

/privacy-policy

Create privacy policy.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "title": "Privacy Policy",
  "description": "...",
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Privacy Policy created successfully",
  "data": {
    "id": "...",
    "title": "Privacy Policy",
    "description": "...",
  }
}
```

/privacy-policy

List privacy policies.

Access: Public

Request

JSON / QUERY

```
{
  "query": "page, limit, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Privacy Policies retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```


56. GET

/privacy-policy/:id

Get one privacy policy.

Access: Public

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Privacy Policy retrieved successfully",
  "data": {
    "id": "...",
    "title": "Privacy Policy",
    "description": "..."
  }
}
```

/privacy-policy/:id

Update privacy policy.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "description": "Updated text"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Privacy Policy updated successfully",
  "data": {
    "id": "...",
    "description": "Updated text"
  }
}
```

/privacy-policy/:id

Delete privacy policy.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Privacy Policy deleted successfully",
  "data": {
    "id": "...
  }
}
```

Apple Subscription

8 endpoints in this module.

- 59. POST /subscription/verify — Verify StoreKit 2 purchase.
- 60. GET /subscription/status — Get Apple-verified premium status.
- 61. POST /subscription/restore — Restore Apple purchase history.
- 62. GET /subscription/history — Get verified Apple renewal history.
- 63. POST /subscription/notifications/test — Request Apple test notification.
- 64. POST /subscription/notifications/history — Request Apple notification delivery history.
- 65. GET /subscription/notifications/history/:notificationId — Get locally processed notification metadata.
- 66. POST /apple/webhook — App Store Server Notifications V2 receiver.

/subscription/verify

Verify StoreKit 2 purchase.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "transactionId": "100000123456789",
  "productId": "premium_monthly"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Apple subscription verified successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

Integration note

All /subscription routes are also mounted under the legacy /subscriptions prefix. New Flutter code should use /subscription.

/subscription/status

Get Apple-verified premium status.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Subscription status retrieved successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

/subscription/restore

Restore Apple purchase history.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "originalTransactionId": "100000123456789"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Apple subscription restored successfully",
  "data": {
    "premium": true,
    "expiresAt": "2026-09-03T10:30:00.000Z"
  }
}
```

/subscription/history

Get verified Apple renewal history.

Access: Bearer · USER/ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Apple subscription history retrieved successfully",
  "data": [
    {
      "purchaseDate": "2026-08-03T10:30:00.000Z",
      "expiresAt": "2026-09-03T10:30:00.000Z",
      "productId": "premium_monthly",
      "transactionId": "...",
      "originalTransactionId": "...",
      "environment": "Production",
      "revoked": false
    }
  ]
}
```


/subscription/notifications/test

Request Apple test notification.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "environment": "Sandbox | Production"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Apple test notification requested",
  "data": {
    "testNotificationToken": "..."
  }
}
```

/subscription/notifications/history

Request Apple notification delivery history.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "environment": "Sandbox",
  "paginationToken": "optional",
  "startDate": 1785686400000,
  "endDate": 1785772800000,
  "notificationType": "optional",
  "notificationSubtype": "optional",
  "transactionId": "optional",
  "onlyFailures": false
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Apple notification history retrieved",
  "data": {
    "notificationHistory": [],
    "hasMore": false
  }
}
```

/subscription/notifications/history/:notificationId

Get locally processed notification metadata.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Apple notification details retrieved",
  "data": {
    "notificationUUID": "...",
    "notificationType": "DID_RENEW",
    "processedAt": "2026-08-03T10:30:00.000Z"
  }
}
```

/apple/webhook

App Store Server Notifications V2 receiver.

Access: Apple server only

Request

JSON / QUERY

```
{
  "signedPayload": "<Apple JWS>"
}
```

Success response

HTTP

200 OK text

Integration note

Flutter must never call this endpoint.

Administration

5 endpoints in this module.

- 67. GET /admin/dashboard — Get revenue/user/subscription dashboard.
- 68. GET /admin/subscribers — List premium subscribers.
- 69. GET /admin/monthly-revenue — Get last 12 months revenue.
- 70. DELETE /admin/delete-account/:id — Delete user and related records.
- 71. PATCH /admin/update-user/:id — Update user fields.

/admin/dashboard

Get revenue/user/subscription dashboard.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Dashboard data retrieved successfully",
  "data": {
    "totalRevenue": 1000,
    "totalActiveUsers": 25,
    "totalSubscribers": 10,
    "newSubscribersLast60Days": 3,
    "subscriptionDistribution": []
  }
}
```

/admin/subscribers

List premium subscribers.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "query": "page, limit, searchTerm, sortBy, sortOrder"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "Subscribers retrieved successfully",
  "pagination": {
    "page": 1,
    "limit": 10,
    "totalPage": 1,
    "total": 1
  },
  "data": [
    {
      "id": "...",
      "createdAt": "2026-08-03T10:30:00.000Z"
    }
  ]
}
```

/admin/monthly-revenue

Get last 12 months revenue.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Monthly revenue retrieved successfully",
  "data": [
    {
      "year": 2026,
      "month": 8,
      "revenue": 299
    }
  ]
}
```


/admin/delete-account/:id

Delete user and related records.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

No request body.

Success response

JSON

```
{
  "success": true,
  "message": "Account and all related data deleted successfully",
  "data": {
    "id": "...
  }
}
```

/admin/update-user/:id

Update user fields.

Access: Bearer · ADMIN/SUPER_ADMIN

Request

JSON / QUERY

```
{
  "name": "Updated Name",
  "status": "active"
}
```

Success response

JSON

```
{
  "success": true,
  "message": "User updated successfully",
  "data": {
    "id": "...",
    "name": "Updated Name",
    "status": "active"
  }
}
```

Flutter implementation checklist

- Configure API base URL per flavor; never hard-code production URLs in widgets.
- Store access and refresh tokens in secure storage.
- Attach Bearer access JWT only to protected routes.
- Refresh authentication once on 401; avoid infinite retry loops.
- Parse ISO timestamps as UTC and format only in the UI layer.
- Use typed response models and handle data being null or an empty array.
- Use multipart requests for profile, income, expense, notices, scan, OCR, and chat attachments.
- Treat report responses as bytes/files, not JSON.
- Hide admin routes from regular-user builds and enforce roles server-side.
- Never call /apple/webhook from Flutter or ship Apple server credentials.
- Log request IDs/statuses safely; never log passwords, JWTs, or reset tokens.